МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий
Кафедра «Инфокогнитивные технологии»

КУРСОВОЙ ПРОЕКТ

на тему: *«Основы объектно-ориентированного программирования. Реализация игры-симулятора «Жизнь»*

Направление подготовки 09.03.03 «Прикладная информатика»
Профиль «Разработка и интеграция бизнес-приложений»

Выполнила:

Андрианова Мария Ивановна

Москва 2025

Введение

Игра «Жизнь», придуманная английским математиком Джоном Конвеем в 1970 г., – это клеточный автомат, представляющий собой дискретную динамическую систему [1]. Его используют для моделирования процессов, происходящих при зарождении, развитии и гибели колонии живых организмов.

Действие игры происходит на бесконечной плоскости, разделенной на клетки. Каждая клетка имеет восемь соседей и может находиться в одном из двух состояний – живом или мертвом. Основная идея игры состоит в том, чтобы, начав с некоторой конфигурации живых клеток, проследить за ее эволюцией.

Начальная конфигурация живых клеток определяется игроком. Каждое следующее поколение колонии рассчитывается на основе предыдущего по следующим правилам («генетическим законам»):

1) в пустой (мертвой) клетке зарождается жизнь, если с ней соседствуют три живые клетки;
2) живая клетка продолжает жить, если с ней соседствуют две или три живые клетки;
3) живая клетка умирает от одиночества, если у нее меньше двух живых соседей;
4) живая клетка умирает от перенаселенности, если у нее больше трех.

Игра прекращается, если:

1) на поле не останется ни одной живой клетки;
2) конфигурация на очередном шаге в точности (без сдвигов и поворотов) повторит себя же на одном из более ранних шагов;
3) при очередном шаге ни одна из клеток не меняет своего состояния.

Часто исходные конфигурации клеток переходят в колебательный режим или обретают устойчивые состояния. Примером периодически повторяющейся фигуры со смещением может служить «планер» (glider), его особенностью является способность спонтанно возникать и перемещаться за пределы видимого пространства (рис. 1).

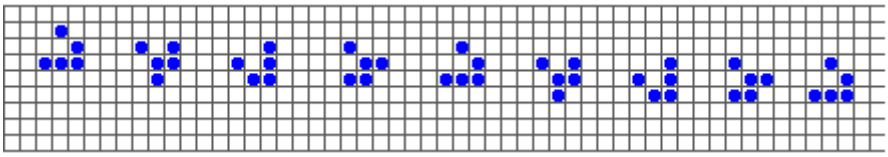


Рисунок 1 – Эволюция объекта «планер»

1. Постановка задачи

Целью настоящей работы является изучение основных свойств и преимуществ объектно-ориентированного программирования посредством реализации симулятора игры «Жизнь» на языке программирования Java [2].

Требования к программному решению:

- Управление приложением должно осуществляться вводом команд из консоли.
- Состояние игрового поля должно демонстрироваться в окне, созданном с помощью графической библиотеки Swing.
- Размерность поля (число строк, число столбцов) должна задаваться пользователем.
- Верхнюю границу поля следует считать продолжением нижней, а левую границу – продолжением правой.
- Исходные колонии клеток должны задаваться текстовыми файлами (пример представлен на рисунке 2).
- Пользователю должна быть предоставлена возможность указать путь к файлу с описанием фигуры (или колонии) и ее местоположение на поле в виде координат левого верхнего угла минимального прямоугольника, ограничивающего фигуру.
- Структура кода должна отвечать парадигме объектно-ориентированного программирования (рекомендуется как минимум создание классов Field и Cell).
- Программа не должна содержать графические элементы управления и обрабатывать события.

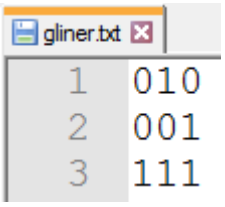


Рисунок 2 – Пример описания конфигурации живых клеток

2. Проектирование и разработка приложения

Проект состоит из 6 классов, каждый из которых выполняет свою задачу в реализации игры «Жизнь». В них находятся публичные статические методы, вызываемые из других классов без создания экземпляра класса, публичные методы экземпляра класса, доступные из любого места программы и вызываемые с помощью объекта определённого класса и приватные методы экземпляра класса, которые могут быть вызваны только внутри самого класса. Также в качестве атрибутов класса выступают публичные статические константы, доступные для использования в любом классе программы и имеющие неизменяемое значение после инициализации, приватные переменные, доступные только внутри самого класса, в котором они были объявлены. Рассмотрим подробнее каждый класс:

Класс Main. Данный класс является основным в программе. В нём находится публичный статический метод rulesGame(), который не возвращает значение, а только выводит текстовое сообщение, подробно описывающее игру «Жизнь», её правила и подготовку к запуску игры. Также класс содержит публичный статический метод main(), который является точкой входа для запуска программы. В нём используется конструкция throws IOException для обработки исключений, связанных с операциями ввода-вывода, она была необходима для корректной работы программы при чтении пользовательского файла с описанием фигуры (или колонии). В этом методе создаются и инициализируются такие объекты классов, как scanner класса Scanner для работы с вводом данных с клавиатуры, field класса Field для управления полем игры, window класса GraphicWindow для отображения графики, evolution класса Evolution для управления эволюцией клеток и gameCompletion класса GameCompletion для проверки завершения игры. Также в

нём у пользователя запрашиваются количество строк и столбцов игрового поля, путь к файлу с начальной конфигурацией, номер строки и столбца расположения её левого верхнего угла на поле. Именно в этом методе происходит управление игрой с помощью команд и дальнейшее завершение игры.

Класс GraphicWindow. Данный класс отвечает за графическое отображение игрового поля в окне. Он использует графическую библиотеку Swing для создания окна и работы с графикой. Класс наследует все свойства и методы от класса JFrame. В качестве атрибутов класса выступают публичные статические константы WIDTH и HEIGHT – ширина и высота графического окна, объект field класса Field с модификатором доступа private. В классе представлен конструктор данного класса, который принимает объект типа Field. Внутри конструктора устанавливаются размеры окна с помощью метода setSize(), устанавливается заголовок окна с помощью метода setTitle(), окно становится видимым с помощью метода setVisible(true), задаётся начальное положение окна через метод setLocation(), вызов метода setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) позволяет завершить программу при закрытии окна, а вызов метода setAlwaysOnTop(true) делает окно всегда поверх других. Также в классе находится переопределённый метод paint(Graphics g), в котором вызывается метод super.paint(g) для очистки окна, рассчитываются координаты, чтобы центрировать игровое поле на экране и вызываются методы для отрисовки сетки игрового поля и живых клеток. Класс GraphicWindow играет важную роль в создании интерфейса игры, позволяя пользователю наблюдать за развитием клеток.

Класс Cell. Данный класс Cell представляет клетку игры "Жизнь". В нём содержится приватная переменная cell типа int, которая хранит состояние клетки (0 - мёртвая, 1 - живая). Класс имеет конструктор для инициализации состояния клетки, а также публичные методы экземпляра класса - геттер и сеттер для получения и изменения состояния клетки соответственно. Публичный метод экземпляра класса draw() имеет параметры: Graphics g - объект для рисования на экране, int x и int y - координаты, указывающие положение клетки, int sizeCell - размер клетки. Этот

метод отвечает за отрисовку клетки на экране, задавая её цвет и рисуя в виде круга заданного размера. Этот класс является основой для управления клетками в игре.

Класс Field. Данный класс представляет игровое поле для симуляции игры «Жизнь». Он включает атрибуты, а именно приватные переменные: двумерный массив клеток класса Cell для хранения состояния клеток на всём игровом поле, rows и cols типа целочисленного типа для указания размера поля, а также sizeCell типа int для отображения размера клетки. Представлен конструктор данного класса, который принимает объект количество строк и столбцов. Внутри конструктора происходит инициализация атрибутов rows, cols, cells и заполнение двумерного массива мёртвыми клетками. Геттеры getRows(), getCols(), getSizeCell(), getCells(int row, int col) обеспечивают доступ количеству строк, количеству столбцов, размеру клетки и массиву клеток. Сеттер setCells(int row, int col, int state) обеспечивает доступ для изменения массива клеток. Также в классе представлены публичные методы экземпляра класса: drawField(), отвечающий за рисование сетки игрового поля, drawLivingCells(), рисующий только живые клетки на поле и loadingShape(), который загружает конфигурацию клеток из файла на определённое место в игровом поле. Класс обеспечивает основные функции для создания и визуализации игрового поля.

Класс Evolution. Данный класс отвечает за управление процессом эволюции клеток игрового поля в симуляции игры «Жизнь». Он включает атрибут field класса Field с модификатором доступа private, представляющий игровое поле. Конструктор принимает объект игрового поля и присваивает его атрибуту. Основной публичный метод экземпляра класса evolve() выполняет один шаг эволюции, обновляя состояние клеток согласно правилам игры. Для подсчёта живых соседей клетки используется метод экземпляра класса countLiveNeighbors(). Класс определяет логику обновления состояния игрового процесса.

Класс GameCompletion. Данный класс отвечает за контроль завершения игры «Жизнь». Его атрибуты с модификатором доступа private включают объект field класса Field (игровое поле) и список ArrayList с объектами типа Cell[][], имеющий название history, в котором хранятся состояния поля на каждом шаге. Основной

публичный метод экземпляра класса checkingCompletionGame() проверяет три условия завершения игры с помощью приватных методов экземпляра класса, которые возвращают логическое значение true или false: deadCells() - отсутствие живых клеток, repeatConfigurationEarlier() - повторение конфигурации на одном из более ранних шагов и configurationNotChanged() - отсутствие изменений в конфигурации. Для проверки идентичности массивов используется приватный метод экземпляра класса identicalArrays(). Класс помогает отслеживать, когда игра должна быть завершена.

Взаимодействие объектов классов в проекте строится на основе чётко распределённых ролей, которые позволяют моделировать игру «Жизнь» в соответствии с её правилами. Каждый класс не только выполняет свою задачу, но и обменивается данными с другими классами через методы, объекты и параметры. Рассмотрим взаимодействие между всеми объектами подробнее:

Cell и Field. Класс Field выполняет роль главного контейнера для объектов класса Cell. Каждая клетка, представленная объектом Cell, хранит своё состояние. Для управления этим состоянием класс Cell предоставляет методы, такие как getCell() для получения текущего состояния клетки и setCell() для её изменения. При создании объекта Field инициализируется двумерный массив клеток (Cell[][] cells). Все клетки изначально задаются как мёртвые. Объект класса Field является центральным объектом, на котором строится вся структура игры.

Field и Evolution. Класс Evolution управляет логикой игры, используя объект Field для хранения и обновления состояния клеток. Он получает текущее состояние клеток поля через метод getCells(), анализирует их соседей и применяет правила игры. После вычислений класс Evolution обновляет состояние клеток, создавая новый массив клеток и передавая его в Field через метод setCells(). Таким образом, Field хранит актуальное состояние поля, а Evolution управляет изменениями клеток на каждом шаге игры.

Field и GameCompletion. Класс GameCompletion использует объект Field для анализа текущего состояния игрового поля и проверки условий завершения игры.

Он обращается к методам getCells() для получения актуального состояния клеток и сравнивает его с предыдущими конфигурациями, хранящимися в истории. Если текущая конфигурация совпадает с одной из ранее сохранённых, либо если на поле нет живых клеток или состояние клеток не изменилось, игра завершается. Таким образом, Field предоставляет данные о состоянии клеток, а GameCompletion использует эти данные для принятия решения о завершении игры.

Field и GraphicWindow. Объект класса Field передаёт свои данные в GraphicWindow для визуализации игрового поля. С помощью метода drawField() объект класса Field рисует сетку, вычисляя размеры клеток и их координаты. Метод drawLivingCells() обрабатывает клетки, которые находятся в состоянии "живая", и передаёт команды на их отображение через метод draw() объекта класса Cell. Таким образом, Field предоставляет данные о состоянии клеток, а GraphicWindow отвечает за их графическое отображение.

Main и остальные классы. Класс Main управляет всеми компонентами игры, связывая их между собой. Он создаёт экземпляры классов Field, Evolution, GameCompletion и GraphicWindow, и организует их взаимодействие. В игровом цикле Main вызывает метод evolve() из класса Evolution, чтобы обновить состояние клеток, метод checkingCompletionGame() из класса GameCompletion, чтобы проверить условия завершения игры, а также методы отображения из GraphicWindow, чтобы обновить визуализацию игрового поля. Этот процесс обеспечивает пошаговое развитие игры, координируя работу всех классов.

В игре "Жизнь" важнейшими являются методы, которые обеспечивают изменение состояния клеток на поле, проверку условий завершения игры, а также загрузку конфигурации с внешнего файла. Эти методы реализуют основную логику симуляции, включая эволюцию клеток, подсчёт живых соседей, проверку различных условий завершения игры и возможность загрузки заранее определённых конфигураций. Рассмотрим алгоритмические аспекты следующих методов:

Метод loadingShape() класса Field предназначен для загрузки фигуры (или колонии) пользователя на игровое поле. В качестве параметров принимает файл и

начальные координаты для выгрузки фигуры на поле. Открывается текстовый файл, содержащий конфигурацию клеток, далее идёт считывание данных построчно. Каждая строка является последовательностью символов: "1" для живой клетки и "0" для мёртвой. Для каждого символа строки рассчитываются координаты соответствующей клетки на поле с учётом начальных координат и с учётом, что верхняя граница поля считается продолжением нижней, а левая граница - продолжением правой (с помощью использования оператора остатка от деления). Клетки устанавливаются в соответствии с конфигурацией, используя метод setCell(). Также учитывается случай, когда последняя строка не имеет символа переноса строки. После обработки всех строк файл закрывается. Фрагмент кода:

```
public void loadingShape(String file, int startRow, int startCol) throws IOException {
    FileReader fileReader = new FileReader(file); // Открытие файла
    int row = startRow; // Номер текущей строки
    int symbol; // Переменная для считывания символов
    String str = ""; // Переменная для хранения строки

    while ((symbol = fileReader.read()) != -1) {
        if (symbol == '\n') {
            for (int col = 0; col < str.length(); col++) {
                // Учёт бесконечности поля
                int newRow = (row % rows);
                int newCol = (startCol + col) % cols;

                if (str.charAt(col) == '1') {
                    cells[newRow][newCol].setCell(1);
                }
            }
            row++;
            str = "";
        } else {
            str += (char) symbol;
        }
    }

    // Обрабатываем последнюю строку, если она не оканчивается символом новой строки
    if (!str.isEmpty()) {
        for (int col = 0; col < str.length(); col++) {
            // Учёт бесконечности поля
            int newRow = (row % rows);
            int newCol = (startCol + col) % cols;

            if (str.charAt(col) == '1') {
                cells[newRow][newCol].setCell(1);
            }
        }
    }
```

Метод evolve() класса Evolution отвечает за обновление состояния клеток на поле. Метод эволюции начинается с создания нового массива клеток, где каждая клетка изначально имеет состояние "мертвая". Затем для каждой клетки подсчитываются живые соседи, и на основе этих данных принимается решение о её будущем состоянии. Если клетка живая и имеет менее двух или более трёх живых соседей, она умирает, а если клетка мёртвая и окружена тремя живыми клетками, она становится живой. После расчёта всех клеток новый массив используется для обновления поля. Фрагмент кода:

```
public void evolve() {
    int rows = field.getRows();
    int cols = field.getCols();
    Cell[][] cells = field.getCells();
    Cell[][] newCells = new Cell[rows][cols];

    // Создаём объекты Cell для каждой клетки в новом массиве
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            newCells[i][j] = new Cell(0); // Инициализация каждой клетки
        }
    }

    // Проходим по каждой клетке поля
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            int liveNeighbors = countLiveNeighbors(i, j);
            int currentCell = cells[i][j].getCell();

            // Применяем правила игры
            if (currentCell == 1) {
                if (liveNeighbors < 2 || liveNeighbors > 3) {
                    newCells[i][j].setCell(0); // Умирает от одиночества или перенаселённости
                } else {
                    newCells[i][j].setCell(1);
                }
            } else {
                if (liveNeighbors == 3) {
                    newCells[i][j].setCell(1); // В мёртвой клетке зарождается жизнь
                } else {
                    newCells[i][j].setCell(0);
                }
            }
        }
    }

    field.setCells(newCells);
}
```

Метод countLiveNeighbors() класса Evolution используется для подсчёта количества живых соседей для клетки на поле. Принимает на вход номер строки и столбца проверяемой клетки. Метод перебирает соседние клетки вокруг указанной клетки в пределах игрового поля. Он проходит по всем клеткам в области от -1 до 1 по вертикали и горизонтали, исключая саму клетку. При этом учитывается, что поле имеет "бесконечные" границы, и соседние клетки могут располагаться на противоположной стороне поля. Если соседняя клетка живая, счётчик увеличивается. Результатом работы метода является количество живых соседей, которое используется для определения дальнейшего состояния клетки. Фрагмент кода:

```
private int countLiveNeighbors(int row, int col) {
    int liveNeighbors = 0; // Счётчик живых соседей
    int rows = field.getRows();
    int cols = field.getCols();
    Cell[][] cells = field.getCells();

    // Перебираем всех соседей клетки
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            if (i == 0 && j == 0) {
                continue; // Пропускаем саму клетку
            }
            // Учёт бесконечности поля
            int neighborRow = (row + i + rows) % rows;
            int neighborCol = (col + j + cols) % cols;

            // Увеличиваем счётчик, если соседняя клетка живая
            if (cells[neighborRow][neighborCol].getCell() == 1) {
                liveNeighbors++;
            }
        }
    }

    return liveNeighbors;
}
```

Метод deadCells() класса GameCompletion проверяет, есть ли хотя бы одна живая клетка на поле. Он проходит по двумерному массиву клеток игрового поля, проверяя каждую клетку. Если хотя бы одна клетка имеет значение 1 (живая клетка), метод сразу возвращает false, означая, что игра ещё не завершена, потому что есть живые клетки. Если же все клетки в поле мертвы (значение 0 для каждой клетки), метод возвращает true, что означает завершение игры. Фрагмент кода:

```
private boolean deadCells() {
    Cell[][] currentField = field.getCells();
    for (int i = 0; i < field.getRows(); i++) {
        for (int j = 0; j < field.getCols(); j++) {
            if (currentField[i][j].getCell() == 1) {
                return false;
            }
        }
    }
    return true;
}
```

Метод repeatConfigurationEarlier() класса GameCompletion проверяет, повторяется ли текущая конфигурация клеток на поле с одной из предыдущих. Для этого он сравнивает текущее состояние клеток с каждым сохранённым состоянием из истории. В случае, если текущее состояние идентично хотя бы одному из предыдущих состояний, метод возвращает true, что указывает на повторение конфигурации. Если ни одно из сохранённых состояний не совпадает с текущим, метод возвращает false, позволяя продолжить игру. Сравнение конфигураций осуществляется методом identicalArrays(), который проверяет соответствие всех клеток в двух массивах. Фрагмент кода:

```
private boolean repeatConfigurationEarlier() {
    Cell[][] currentField = field.getCells();

    for (Cell[][] previousField : history) {
        if (identicalArrays(previousField, currentField)) {
            return true;
        }
    }
    return false;
}
```

Метод configurationNotChanged() класса GameCompletion проверяет, изменилось ли состояние игрового поля с последнего шага. Для этого он проверяет была ли предыдущая конфигурация и если была, сравнивает текущее состояние клеток с их состоянием на предыдущем шаге. Если оба состояния идентичны (все клетки не изменились), метод возвращает true, что сигнализирует о завершении игры, так как эволюция остановилась. Если состояния различаются, игра продолжается, и метод возвращает false. Сравнение конфигураций осуществляется

методом identicalArrays(), который проверяет соответствие всех клеток в двух массивах. Фрагмент кода:

```
private boolean configurationNotChanged() {
    // Была ли до этой конфигурации предыдущая
    if (history.isEmpty()) {
        return false;
    }

    Cell[][] currentField = field.getCells();
    Cell[][] previousField = history.get(history.size() - 1); // Получаем предыдущую конфигурацию
    return identicalArrays(previousField, currentField);
}
```

Последние три метода являются основными для проверки условий завершения игры. Если хотя бы одно условие выполняется, метод checkingCompletionGame() возвращает true, что приводит к завершению игры.

3. Тестирование и сценарии работы в приложении

Для тестирования программы "Жизнь" необходимо проверить корректность работы ключевых методов: эволюции клеток, завершения игры и загрузки конфигурации из файла.

Проверим корректность работы метода evolve() класса Evolution, отвечающего за процесс эволюции клеток на игровом поле. Рассмотрим выполнение программой следующих условий:

В пустой (мёртвой) клетке зарождается жизнь, если с ней соседствуют три живые клетки. Живая клетка продолжает жить, если с ней соседствуют две или три живые клетки. Для тестирования этого процесса возьмём триплет следующего вида (рис. 3).

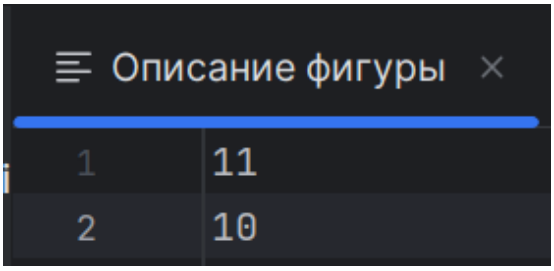


Рисунок 3 – Пример фигуры для первого теста

Было создано поле размером 5 × 5, на которое разместили данную фигуру на координаты (2; 2). Протестировав данную конфигурацию, мы смогли убедиться, что программа корректно выполняет условие зарождения клетки и продолжения её жизни. Клетки, которые находились на поле, продолжили жить, так как каждая имела по две соседних клетки. А на пересечении 3 строки и 3 столбца появилась новая клетка, с которой соседствуют три живые клетки (рис. 4).

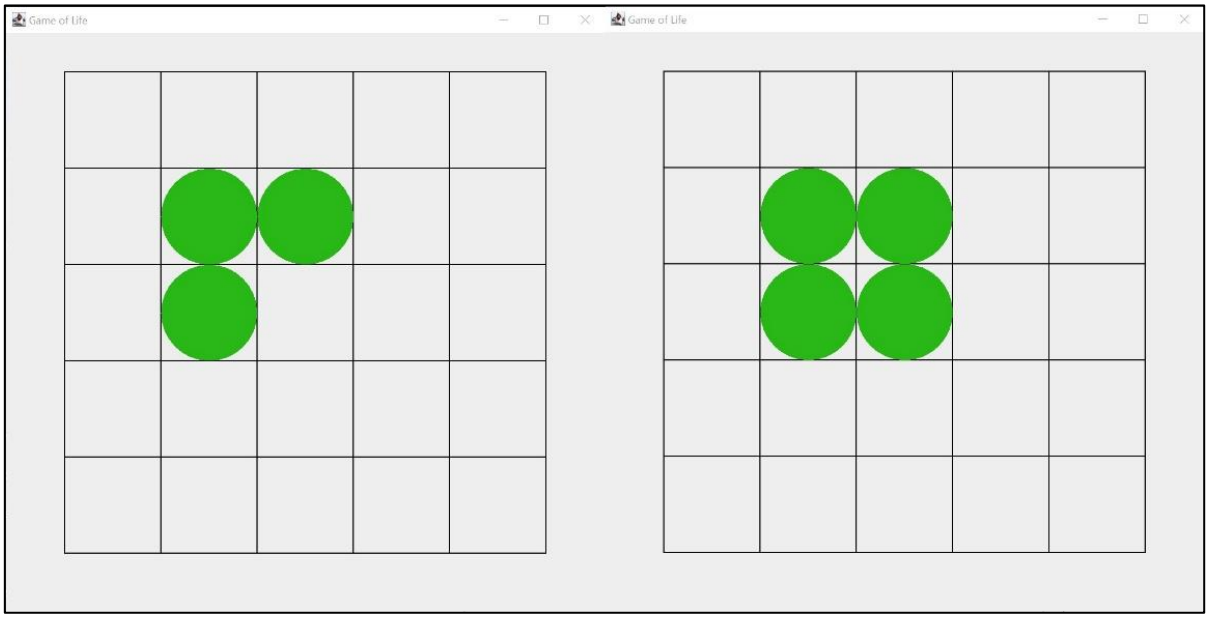


Рисунок 4 – Зарождение новой клетки

Следующим тестом проверим, что живая клетка умирает от одиночества, если у нее меньше двух живых соседей. В качестве примера возьмём следующую конфигурацию (рис. 5).

Описание фигуры

```
1  0000
2  0010
3  0000
4  0100
5  0100
6  0000
```

Рисунок 5 – Пример колонии фигур для второго теста

Было создано поле размером 12 × 8, на которое разместили данную конфигурацию на координаты (2; 4). Протестировав данную конфигурацию, мы смогли убедиться, что программа корректно выполняет условие смерти клетки. Верхняя не имела соседей, а каждая из нижних клеток имела только одного соседа. В итоге все клетки погибли от одиночества, и поле осталось пустым (рис. 6).

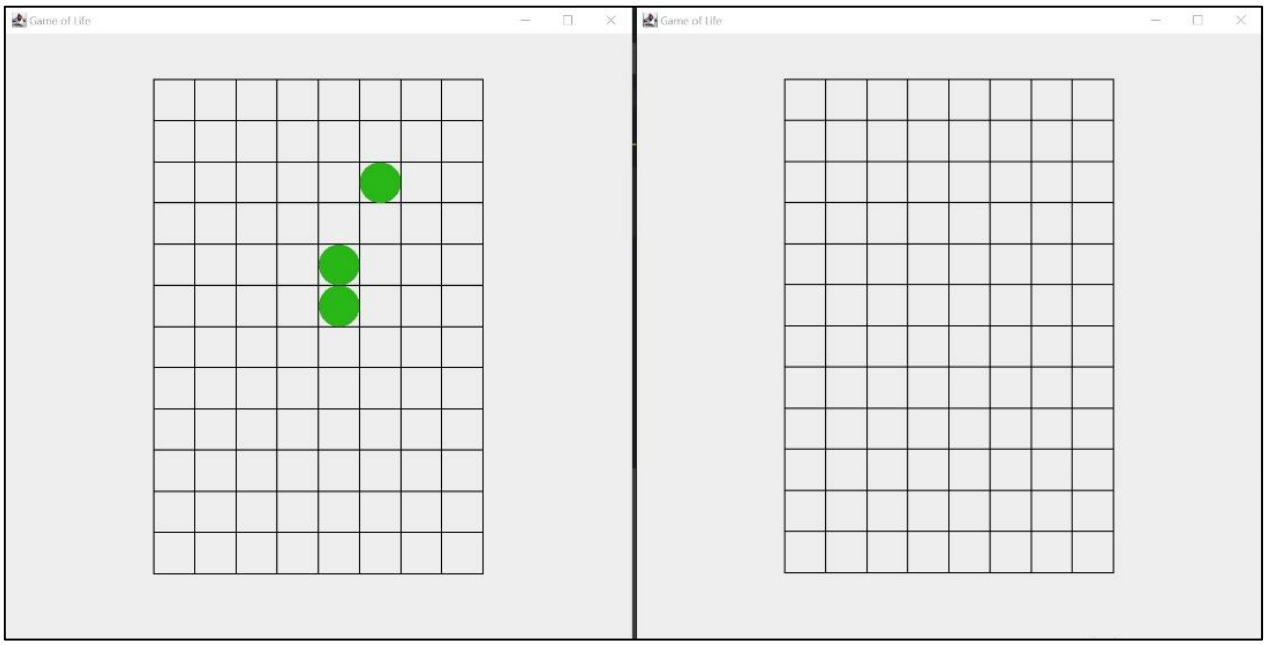


Рисунок 6 – Гибель живых клеток от одиночества

Приведём третий тест на проверку ещё одного условия смерти клети: живая клетка умирает от перенаселённости, если у нее больше трёх живых соседей. Для

этого просто заполним всё поле живыми клетками. Взята следующая конфигурация (рис. 7).

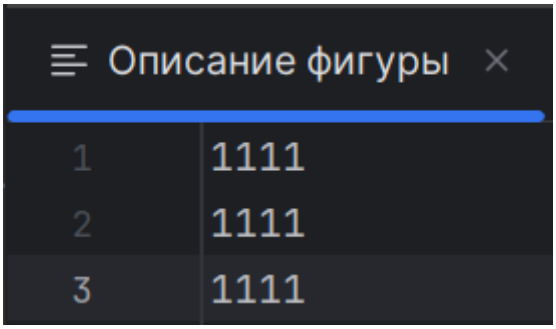


Рисунок 7 – Пример фигуры для третьего теста

Было создано поле размером 3 × 4, на которое разместили данную фигуру на координаты (1; 1). Протестировав данную конфигурацию, мы смогли убедиться, что программа корректно выполняет второе условие смерти клетки. Каждая из клеток имела более трёх соседей, поэтому все клетки погибли от перенаселённости, и поле осталось пустым (рис. 8).

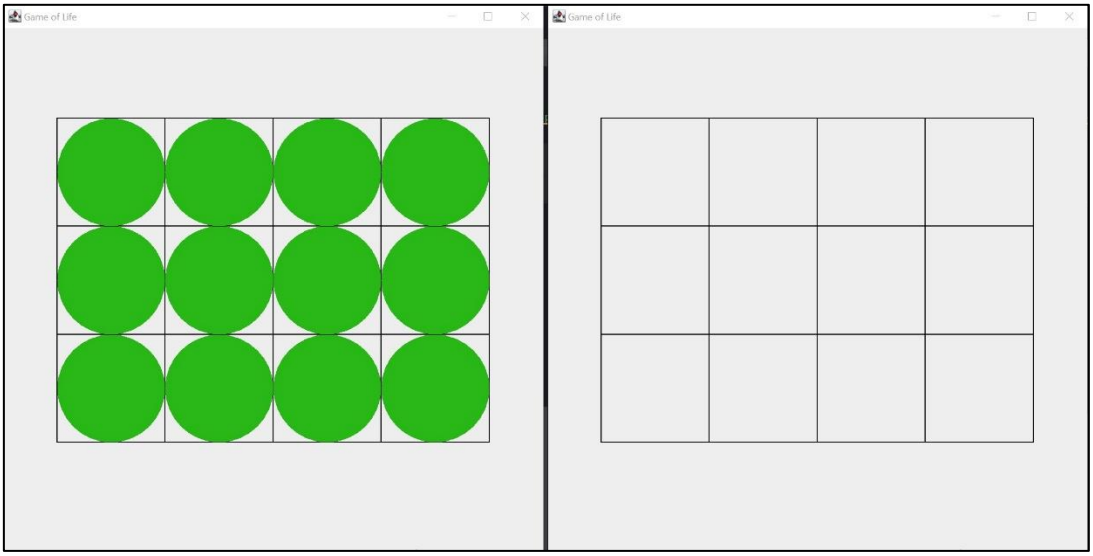


Рисунок 8 – Гибель живых клеток от перенаселённости

Также во время данных тестирований была проверена корректная выгрузка фигур на игровое после с помощью метода loadingShape() класса Field. Потом отметим, что во время первого теста был успешно отработал метод

configurationNotChanged() класса GameCompletion, который проверяет, изменилось ли состояние игрового поля с последнего шага. Так как после первого шага образовалась устойчивая фигура «блок», то на следующем шаге она повторилась и после третьей команды новая отрисовка не произошла, и программа была завершена (рис. 9).

```
Вот и всё, Ваша фигура на поле, а это значит, что можно уже начать игру!
Нажмите Enter для следующего шага или введите 'выход':

Нажмите Enter для следующего шага или введите 'выход':

Нажмите Enter для следующего шага или введите 'выход':

Игра завершена: конфигурация повторилась.
Закройте графическое окно, чтобы завершить работу программы.
```

Рисунок 9 – Завершение игры при первом тестировании

А во время второго и третьего тестирования был успешно отработал метод deadCells() класса GameCompletion, который проверяет, есть ли хотя бы одна живая клетка на поле. Так как в данных тестах после первого шага все клетки умирали, то на после не оставалось ни одной живой клетки и, соответственно, на следующем шаге новая отрисовка не произошла, и программа была завершена (рис. 10).

```
Вот и всё, Ваша фигура на поле, а это значит, что можно уже начать игру!
Нажмите Enter для следующего шага или введите 'выход':

Нажмите Enter для следующего шага или введите 'выход':

Игра завершена: на поле не осталось живых клеток.
Закройте графическое окно, чтобы завершить работу программы.
```

Рисунок 10 – Завершение игры во втором и третьем тестированиях

Теперь рассмотрим полноценную работу программы на завершающем тесте. В качестве начальной конфигурации возьмём «Планер» - это удивительная фигура,

которая постепенно перемещается по игровому полю на одну клетку вправо и на одну клетку вниз. Создан файл с данной конфигурацией (рис. 11).

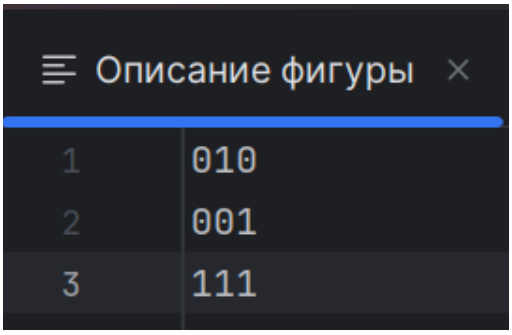


Рисунок 11 – Пример фигуры для четвёртого теста

При запуске программы на экране консоли появляется пояснительное сообщение с описанием игры в «Жизнь» и подготовкой к ней (рис. 12).

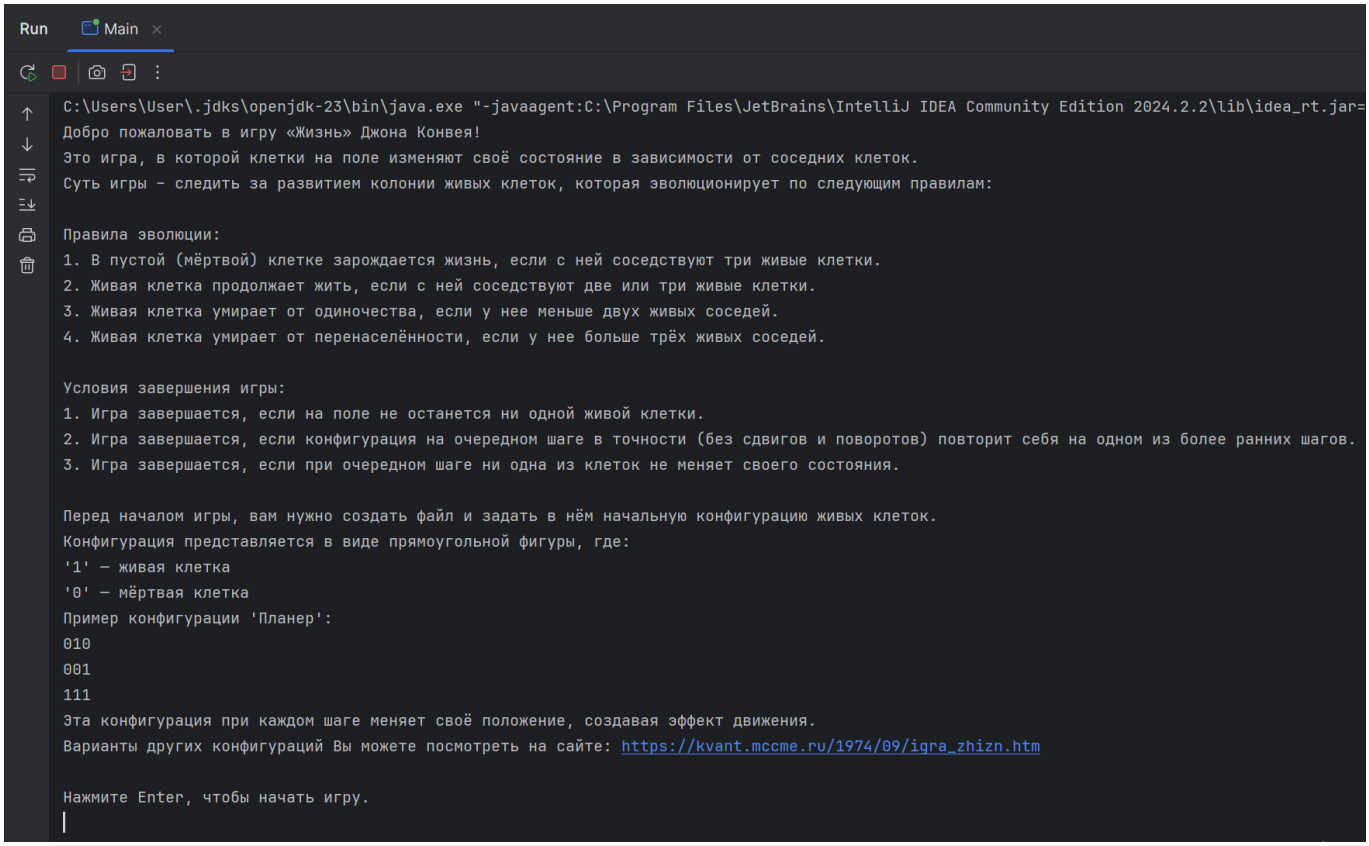


Рисунок 12 – Начало игры, приветственное сообщение в консоли

Далее пользователю предлагают создать поле, указав количество строк и столбцов (для этого теста использовалось поле 7 × 7). После его создания

запрашивается путь к файлу и координаты расположения левого верхнего угла фигуры на игровом поле, взяли (2; 2). Теперь можно начинать игру (рис. 13).

```
Сначала создадим поле для игры.
Введите количество строк: 7
Введите количество столбцов: 7

Поле создано! Теперь разместим на него вашу фигуру (или колонию).
Введите путь к файлу с конфигурацией: C:\Users\User\Desktop\Задания Политех\Основы программирования\The game Life\src\Описание фигуры

Путь к Вашему файлу получен! Теперь нужен номер строки и столбца левого верхнего угла расположения фигуры на поле.
Обратите внимание, что фигура может распасться на части, если окажется за границами поля,
в нашей игре верхняя граница поля считается продолжением нижней, а левая граница – продолжением правой.
Введите номер строки (от 1 до 7): 2
Введите номер столбца (от 1 до 7): 2

Вот и всё, Ваша фигура на поле, а это значит, что можно уже начать игру!
Нажмите Enter для следующего шага или введите 'выход':
```

Рисунок 13 – Подготовка поля к началу игры

После выгрузки фигуры на поле с консоли даются команды для выполнения шагов эволюции, для этого просто нужно нажимать на Enter. Поведение клеток фигуры «Планер» (рис. 14-19).

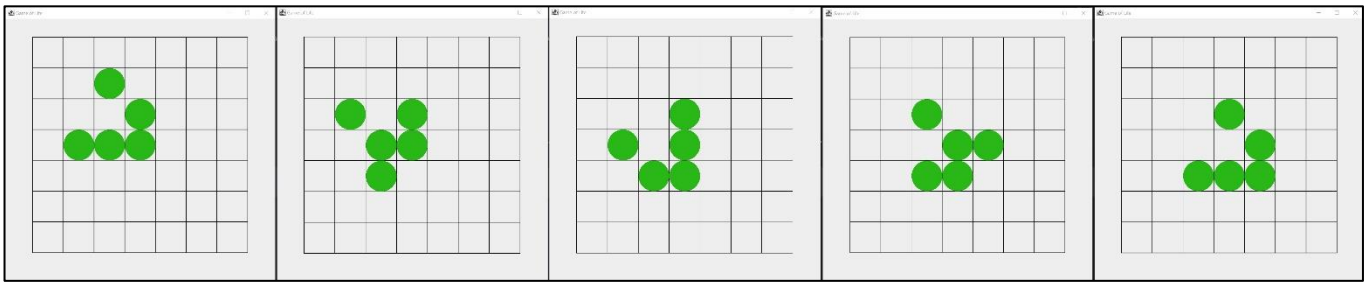


Рисунок 14 – Шаги №0-4

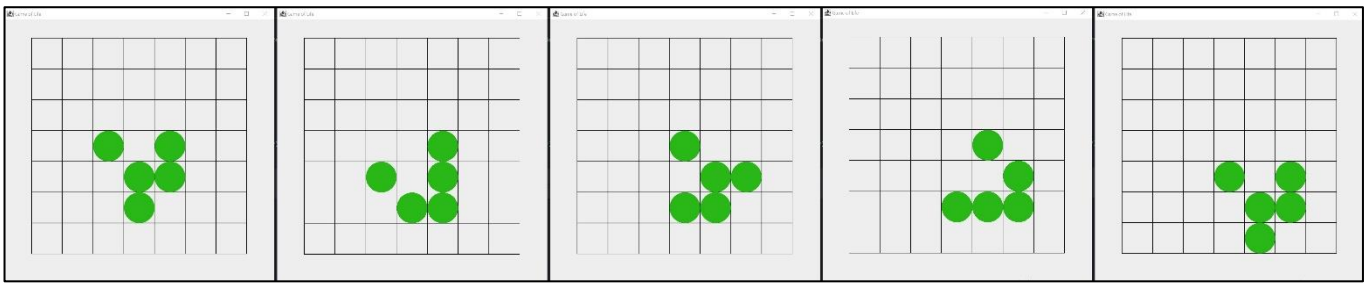


Рисунок 15 – Шаги №5-9

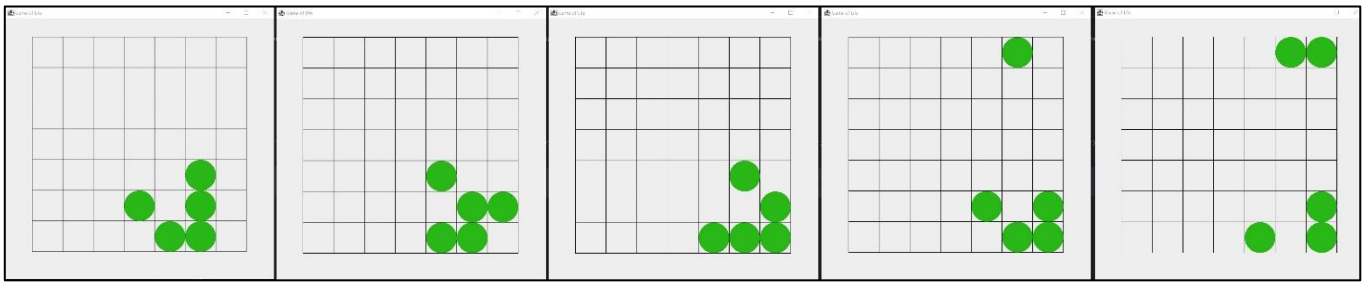


Рисунок 16 – Шаги №10-14

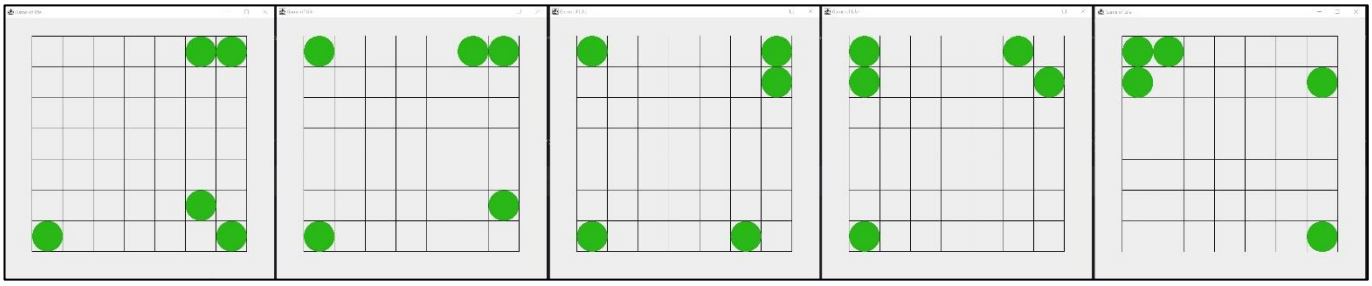


Рисунок 17 – Шаги №15-19

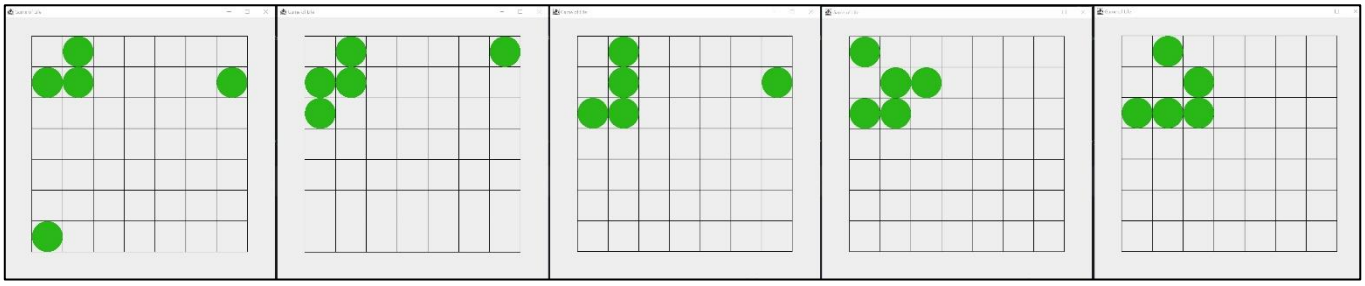


Рисунок 18 – Шаги №20-24

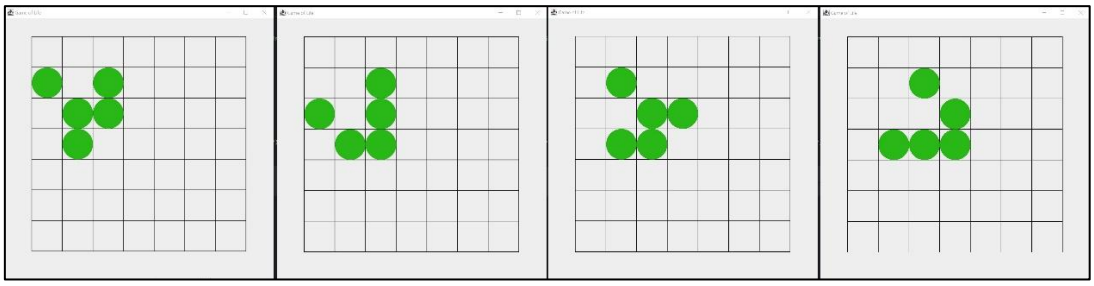


Рисунок 19 – Шаги №25-28

Спустя 28 шагов конфигурация повторилась в точности (без сдвигов и поворотов). Метод repeatConfigurationEarlier() класса GameCompletion отработал корректно и игра завершилась (рис. 20).

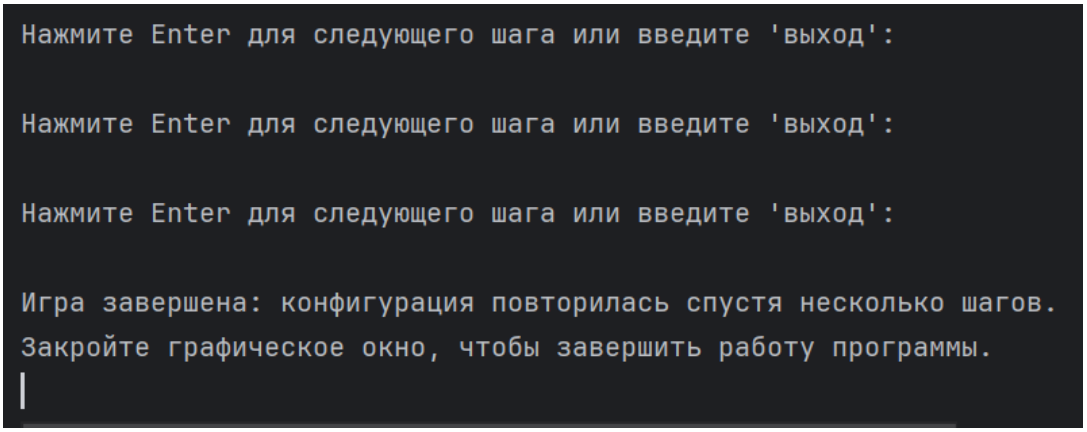


Рисунок 20 – Завершение игры на четвёртом тесте

Также на шагах 13-17 можно увидеть, что метод countLiveNeighbors() класса Evolution выполняет следующее условие: верхняя граница поля считается продолжением нижней, а левая граница - продолжением правой.

В ходе тестирования программы "Жизнь" были проверены ключевые аспекты её функциональности, включая корректность работы методов, отвечающих за эволюцию клеток, завершение игры и загрузку конфигураций из файла. Тестирование подтвердило, что программа корректно реализует основные правила игры: метод evolve() класса Evolution успешно обрабатывает условия зарождения, продолжения жизни и гибели клеток, метод loadingShape() класса Field корректно загружает и размещает фигуры на поле, а методы завершения игры (configurationNotChanged(), deadCells(), repeatConfigurationEarlier()) работают в соответствии с ожидаемым поведением. Также проверена корректность обработки граничных условий методом countLiveNeighbors(). Программа успешно обрабатывает сложные конфигурации, такие как "Планер", и демонстрирует их эволюцию. В результате тестирования подтверждено, что программа стабильно работает и корректно реализует все основные процессы.

Заключение

В рамках курсового проекта по дисциплине «Основы программирования» было разработано отвечающее парадигме объектно-ориентированного программирования приложение, которое моделирует эволюционные изменения колонии живых организмов и может быть использовано как основа для решения различных прикладных задач (например, в области теории информации или компьютерного дизайна).

Список литературы и интернет-ресурсов

1. Клумова И. Игра «Жизнь» [Электронный ресурс]. URL: http://kvant.mccme.ru/1974/09/igra_zhizn.htm (дата обращения: 18.01.2025).

2. Java Tutorials Learning Paths [Электронный ресурс]. URL: https://docs.oracle.com/javase/tutorial/tutorialLearningPaths.html (дата обращения: 18.01.2025).